

High Performance Design

B1: Parallel Processing

B2: Pipelining

B3: Reduction of Clock Skew

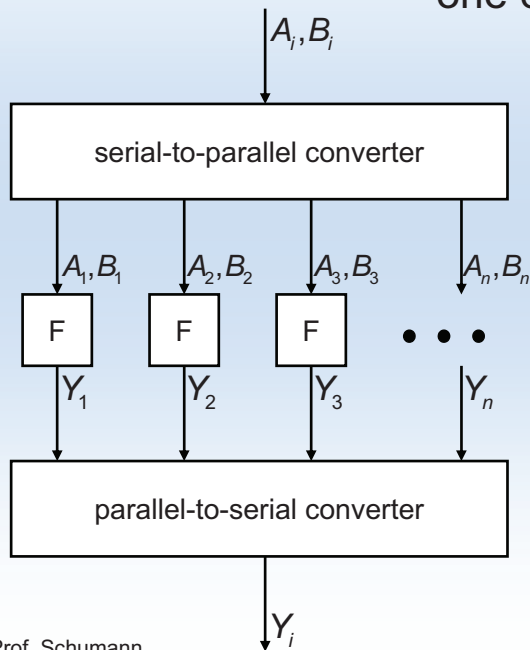
B4: Trends

Parallel Processing

Parallel Processing: increases the sampling rate by replicating hardware so that several inputs can be processed in parallel and several outputs can be produced at the same time

Parallel Processing

Multiple outputs are computed in parallel within one clock cycle



Idea: Multiple implementation of combinatorial logic F

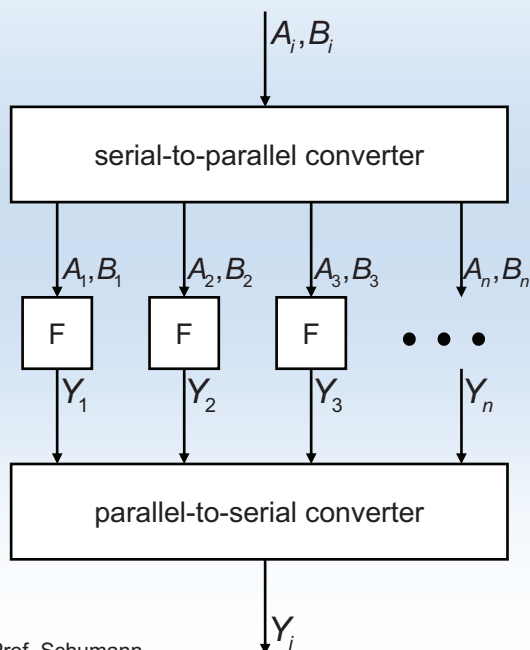
Parallel Processing

With latency τ_F of combinatorial logic F and level N of parallel implementation, the **sampling speed** is increased:

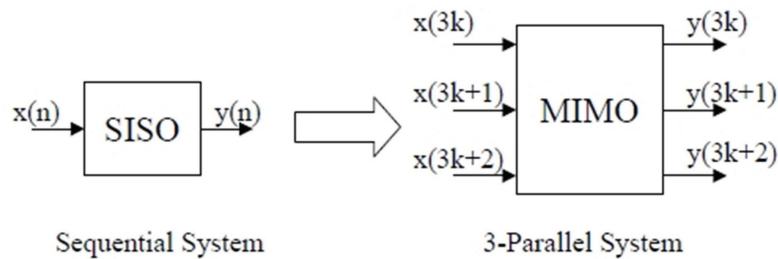
$$f_{parallel} = \frac{N}{\tau_F}$$

But with chip area A_F of logic F, total chip area A is increased:

$$A \approx N \cdot A_F$$



Parallel Processing



- In this parallel processing system, at the k-th clock cycle, 3 inputs $x(3k)$, $x(3k+1)$ and $x(3k+2)$ are processed and 3 samples $y(3k)$, $y(3k+1)$ and $y(3k+2)$ are generated at the output

[3]

SISO=Single_In_Single_Out

MIMO=Multiple_In_Multiple_Out

Parallel Processing

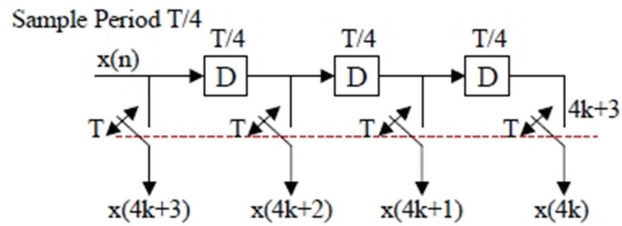
The critical path of the block (or parallel) processing system remains unchanged. But since 3 samples are processed in 1 (not 3) clock cycle, the sample period is given by the following equation:

$$T_{sample} = \frac{T_{clock}}{N} \quad N = 3$$

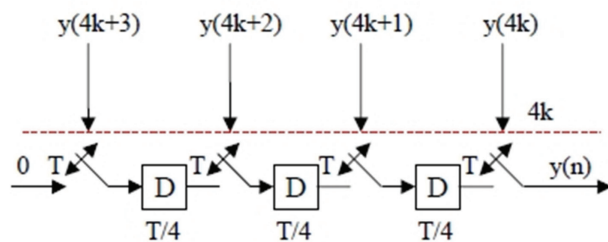
So, it is important to understand that in a parallel system $T_{sample} \neq T_{clock}$, whereas in a pipelined system $T_{sample} = T_{clock}$

Parallel Processing

- A serial-to-parallel converter



- A parallel-to-serial converter



[3]

Parallel Processing

When sample speed does not need to be increased, Parallel Processing can be used for lowering the power consumption!

- In an L-parallel system, the charging capacitance does not change, but the total capacitance is increased by L times
- In order to maintain the same sample rate, the clock period of the parallel circuit is increased to $L \cdot T_{seq}$ (where T_{seq} is the propagation delay of the original sequential circuit).
- Thus the supply voltage can be reduced to $\sigma \cdot V_0$ since there is more time to charge the same capacitance (where V_0 is the supply voltage of the original sequential circuit)
- How to get the supply voltage reduction factor σ ?

Parallel Processing

- Two important formulas:
 - Computing the propagation delay T_{pd} of CMOS circuit

$$T_{pd} = \frac{Q_{charge}}{I_{ds}} = \frac{C_{charge} \cdot V_0}{\frac{\beta}{2} \cdot (V_0 - V_t)^2}$$

C_{charge} : capacitance to be charged or discharged within a single clock cycle

V_0 : supply voltage

V_t : threshold voltage of MOS-transistor

Q_{charge} : total charge necessary to change logical state of output

I_{ds} : drain saturation current

Parallel Processing

- Computing the power consumption in CMOS circuits

$$P_{dynamic} = \alpha \cdot f \cdot C_{total} \cdot V_0^2$$

C_{total} : total capacitance of the CMOS circuit

α : probability that gates changes its logical output state due to clock event

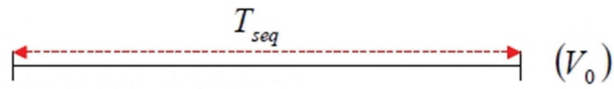
f : clock frequency

V_0 : supply voltage

Parallel Processing

Parallel Processing for Low Power

Sequential(critical path):



Parallel: (critical path when L=3)



- The propagation delay of the original system is still same, but the propagation delay of the L-parallel system is given by [3]

$$L \cdot T_{seq} = \frac{C_{charge} \cdot \sigma \cdot V_0}{\frac{\beta}{2} \cdot (\sigma \cdot V_0 - V_t)^2}$$

Parallel Processing

Parallel Processing for Low Power

- Hence, we get the following equation to compute σ :

$$L \cdot (\sigma \cdot V_0 - V_t)^2 = \sigma \cdot (V_0 - V_t)^2$$

- Once σ is computed, the power consumption of the L-parallel system can be calculated:

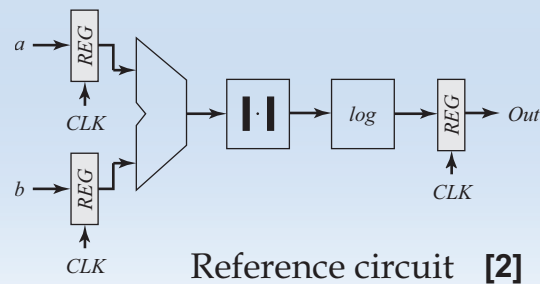
$$P_{parallel} = \alpha \cdot \frac{f}{L} \cdot (L \cdot C_{total}) \cdot (\sigma \cdot V_0)^2 = \sigma^2 \cdot P_{seq}$$

[3]

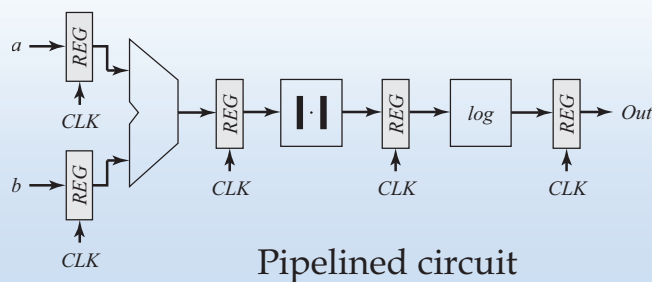
Pipelining

A concept to optimize throughput of a sequential circuit

Ex.: datapath for the computation of $\log(|a + b|)$



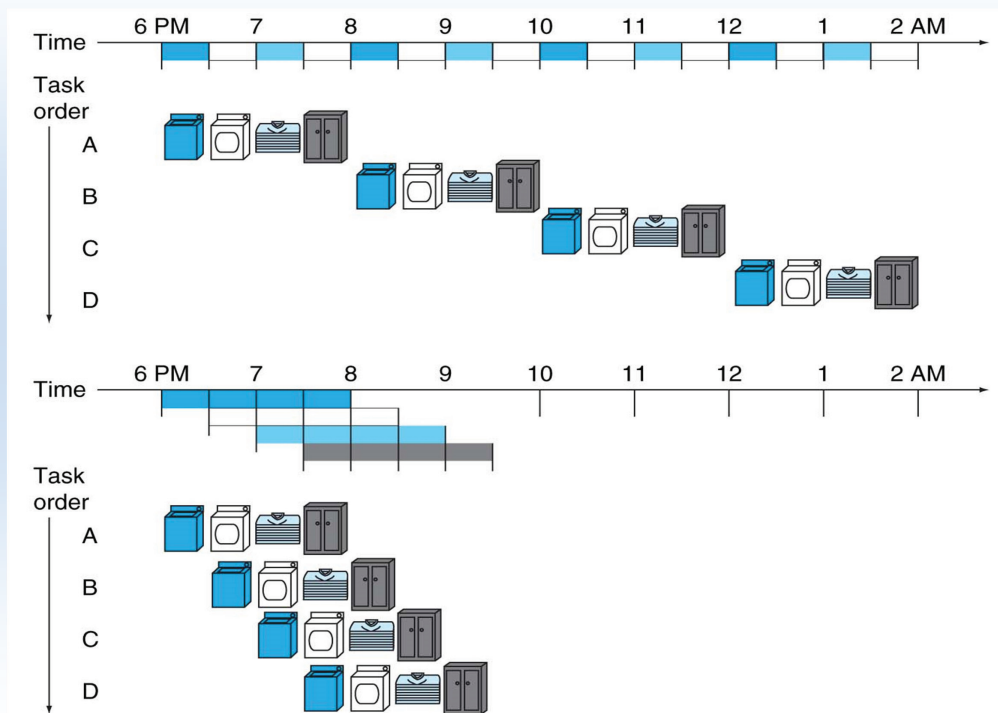
Pipelining



Clock Period	Adder	Absolute Value	Logarithm
1	$a_1 + b_1$		
2	$a_2 + b_2$	$ a_1 + b_1 $	
3	$a_3 + b_3$	$ a_2 + b_2 $	$\log(a_1 + b_1)$
4	$a_4 + b_4$	$ a_3 + b_3 $	$\log(a_2 + b_2)$
5	$a_5 + b_5$	$ a_4 + b_4 $	$\log(a_3 + b_3)$

[2]

Pipelining



Prof. Schumann

Digital System Design

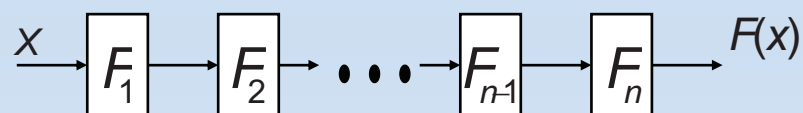
62

Pipelining

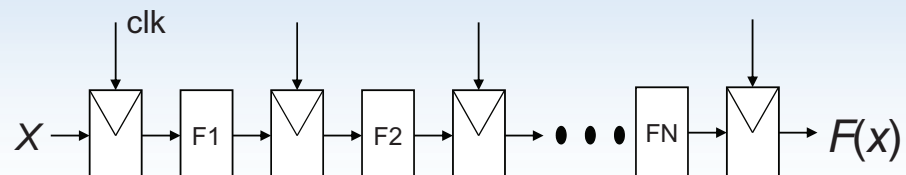
If it is possible to split the operation F in N serial operations $F_1, F_2, \dots, F_N$,

$$F = F_N(F_{N-1}(\dots F_2(F_1)))$$

the operation F can be implemented as following:



Pipelining: place flip-flops between each serial operation!



Prof. Schumann

Digital System Design

63

Pipelining

Computation is performed in a assembly-line fashion - hence the name *pipeline*!

Constraint on the minimum clock period T is given by the serial operation with the maximum propagation delay of all serial operations!

$$T \geq \max(t_{pd,F1}, t_{pd,F2} \dots t_{pd,F_{N-1}}, t_{pd,F_N}) + t_{pcq} + t_{setup}$$

Maximum speedup is number N of stages in pipeline!

Pipelining

Pipelining reduces the time critical path, so clock frequency (=sample frequency) can be increased.

Maximum speedup is number N of stages in pipeline (if the propagation delay of each stage is same!)

$$\text{If: } t_{pd,F1} = t_{pd,F2} = \dots t_{pd,F_{N-1}} = t_{pd,F_N}$$

$$\Rightarrow f_{pipelined} = N \cdot f_{non-pipelined} \quad \text{assuming } t_{pcq} = t_{setup} = 0$$

Pipelining

Pipelining shows two main drawbacks:

- Increase in the number of flip-flops
- Increase in system latency

Increased latency: the difference in the availability of the first output data in the pipelined system and the sequential system

Pipelining

When sample speed (clock frequency) does not need to be increased, Pipelining can be used for lowering the power consumption!

- For a M-level pipelined system, its critical path is reduced to 1/M of its original length, and the capacitance to be charged/discharged in a single clock cycle is also reduced to 1/M of its original capacitance.
- If the same clock frequency is maintained, only 1/M of the original capacitance is getting charged/discharged in the same amount of time. Thus the supply voltage can be reduced to $\sigma \cdot V_0$
- Hence the power consumption of the pipelined circuit is:

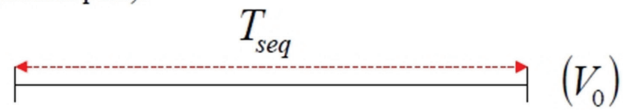
$$P_{pip} = \alpha \cdot f \cdot C_{total} \cdot (\sigma \cdot V_0)^2 = \sigma^2 \cdot P_{seq}$$

How to get the supply voltage reduction factor σ ?

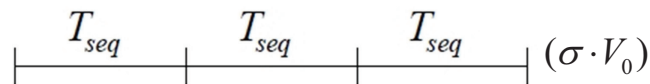
Pipelining

Pipelining for Low Power

Sequential (critical path):



Pipelined: (critical path when M=3)



The propagation delay of the original sequential circuit and the pipelined circuit are:

$$T_{seq} = \frac{C_{charge} \cdot V_0}{\frac{\beta}{2} \cdot (V_0 - V_t)^2} \quad T_{pip} = \frac{(C_{charge} / M) \cdot (\sigma \cdot V_0)}{\frac{\beta}{2} \cdot (\sigma \cdot V_0 - V_t)^2} \quad [3]$$

Pipelining

Pipelining for Low Power

Since the same clock speed is maintained in both designs we get the equation to solve σ :

$$M \cdot (\sigma \cdot V_0 - V_t)^2 = \sigma \cdot (V_0 - V_t)^2 \quad [3]$$

Reduction of Clock Skew

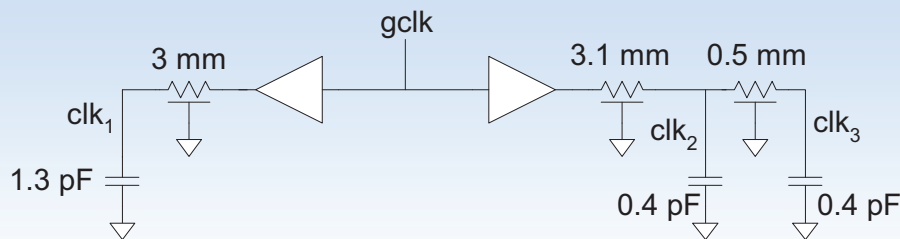
- Synchronous systems use a clock to keep operations in sequence
 - Distinguish *this* from *previous* or *next*
 - Determine speed at which machine operates
- Clock must be distributed to all the sequencing elements
 - Flip-flops and latches
- Also distribute clock to other elements
 - Domino circuits and memories

Clock Distribution

- On a small chip, the clock distribution network is just a wire
 - And possibly an inverter for clkb
- On practical chips, the RC delay of the wire resistance and gate load is very long
 - Variations in this delay cause clock to get to different elements at different times
 - This is called *clock skew*
- Most chips use repeaters to buffer the clock and equalize the delay
 - Reduces but doesn't eliminate skew

Example

- Skew comes from differences in gate and wire delay
 - With right buffer sizing, clk_1 and clk_2 could ideally arrive at the same time.
 - But power supply noise changes buffer delays
 - clk_2 and clk_3 will always see RC skew

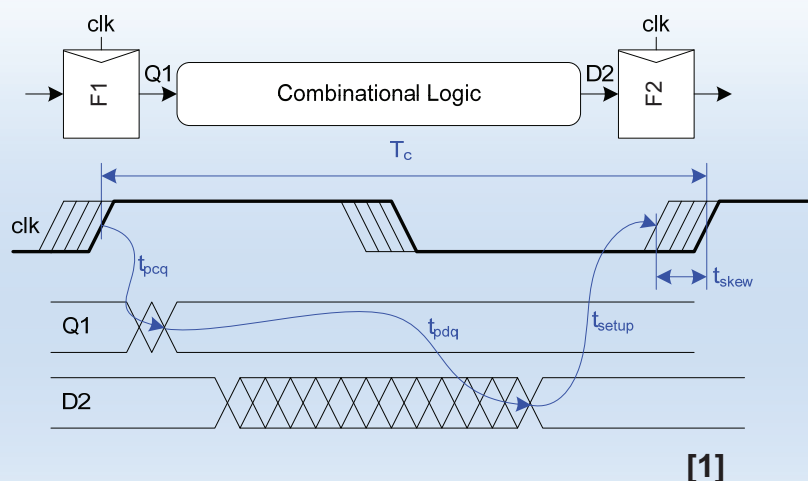


Prof. Schumann

Digital System Design

72

Review: Skew Impact



- Ideally full cycle is available for work
- Skew adds sequencing overhead

$$t_{pd} \leq T_c - \underbrace{(t_{pcq} + t_{setup} + t_{skew})}_{\text{sequencing overhead}} \quad [1]$$

Prof. Schumann

Digital System Design

73

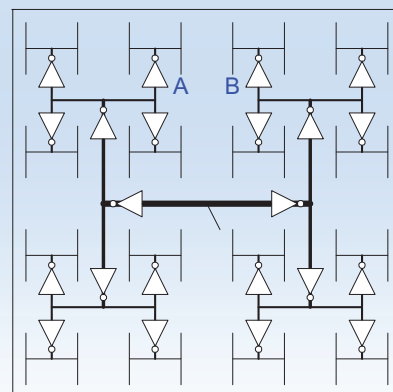
Solutions

Reduce skew by Clock Distribution Networks

- H-tree
- Hybrid

H-Trees

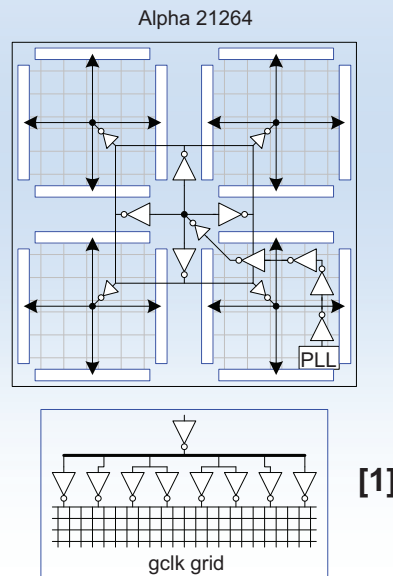
- Fractal structure
 - Gets clock arbitrarily close to any point
 - Matched delay along all paths
- Almost same delay from center to node A and to node B

**[1]**

Hybrid Networks

- Use H-tree to distribute clock to many points
- Tie these points together with a grid

Ex: IBM Power4, PowerPC



Prof. Schumann

Digital System Design

76

Trends

Managing Power and Performance with MPSoC

Workload Candidates by Processing Block

Block	Optimized For:	Ideal For:
APU	• High-level OS support such as Linux • Hypervisor-based computing • Symmetric Multi-Processing and Supervised Asymmetric Multi-Processing	• HMI/UX • Business logic • Network/Cloud interaction
RPU	• Deterministic Operation • Low-Latency Memory Accesses	• Deterministic response time software • Safety-critical software • Standards-compliant software stack (ex: radio)
PL	• Accelerating Applications	• Acceleration / Parallelization • Hardware-assisted effects, transformations, filtering, processing, encoding/decoding ...
GPU	• 2D and 3D Graphics Acceleration under Linux	• Display • Multimedia

[Xilinx: MPSoC Embedded Design Methodology Guide 2017]

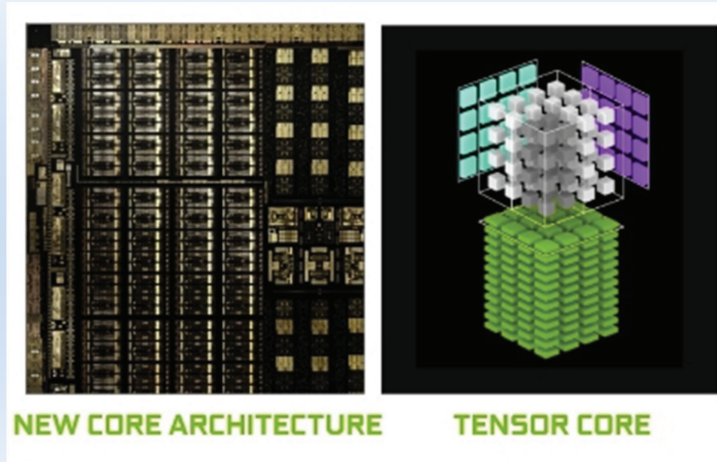
Prof. Schumann

Digital System Design

77

Trends

Specialized Execution Units for Tensor/Matrix Operations



GPU: parallel processing engine for many computationally-intensive applications
(e.g. graphics, deep learning)

[NVIDIA Turing GPU Architecture Whitepaper 2018]